

# Improving deep neural networks classification by preprocessing images

H. Erdmann

*Department of Research and Technology, Boolabs, São Paulo, Brazil*

F.T. Ito

*Department of Computer Science, Universidade Federal de São Carlos, Brazil*

D. Takabayashi

*Department of Research and Technology, Boolabs, São Paulo, Brazil*

D.N. dos Santos

*Department of Research and Technology, Boolabs, São Paulo, Brazil*

*Department of Electrical Engineering, Centro Universitário da FEI, São Paulo, Brazil*

**ABSTRACT:** The amount of images used to train a classifier has a great impact in the algorithm's performance. In the domain of e-commerce, most of the dataset is formed by studio images and we demonstrate in this work that by multiplying the amount of images with different transformations can provide a significant boost on the overall performance of the classifier. Moreover, we list the challenges encountered in such task and present the improvements obtained with a classifier trained with additional images that were synthetically generated by applying several transformations on the original dataset.

## 1 INTRODUCTION

To classify an image correctly is a task with applications in several areas related to digital image processing. It can be done by extracting features that describe the image and compare them with the mean features of the known classes. The simplest classification techniques are binary classifiers, which are used to define if an image belongs or not to a certain class. Moreover, image classification algorithms are usually developed for a specific set of images because the objective of these algorithms is strongly related to their applications. In this work we focus on a multi-classification algorithm using state-of-the-art deep neural networks. It is widely known that the amount of images used to train a classifier has a direct effect on its final results (Fuxman et al. 2009). In the domain of e-commerce, most of the dataset is formed by studio images and we demonstrate how we improved our classifier by multiplying the amount of images with different transformations.

The task of object recognition is a great challenge in computer vision and has been the focus of thorough research by many authors for many years. There are many applications and algorithms towards this goal. Mainly, these applications make use of natural images. There are many image datasets that researchers can use to compare their work, such

as MNIST (Lecun and Cortes, LeCun et al. 2001), Image Net (Deng et al. 2009, Russakovsky et al. 2014) and LabelMe (Russell et al. 2008), composed of, mostly, natural images. However, the common images found in the market and e-commerce sites are doctored ones, with white backgrounds and perfect illumination. Approaches such as Global Contrast Normalization (GCN) (Graham and Sutter 2000) and Zero Components Analysis (ZCA) Whitening (Bell and Sejnowski 1997b, Bell and Sejnowski 1997a) fail in these situations, and cannot be used directly.

Deep Learning (DL) is an area of machine learning that relies on high-level, hierarchized abstractions to create models for specific functions (Krizhevsky et al. 2012a, Schmidhuber 2014). The theory behind Deep Learning gained traction circa 2000/2003, with the advent of powerful graphic cards and new works on the field. One of the prominent algorithms for Image Classification fits the DL profile: the Convolutional Neural Network (Sermanet et al. 2012, Smirnov et al. 2014). In this paper, we discuss the aforementioned approaches to preprocessing and their results in studio images, along with other techniques to expand image datasets and to find meaningful features to use in classifiers. A Convolutional Neural Network is used to get practical results from these preprocessed techniques on an image dataset composed by e-commerce products.

## 2 RELATED WORK

The preprocessing of image data for machine learning application has been extensively covered by several different authors (Kotsiantis and Kanellopoulos 2006, Förstner 2000), as using the image itself pixel by pixel as an instance can be unwieldy at times. A  $256 \times 256$  colored image, for example, yields 196608 intensity values: training a classifier with this data and no Graphic Processing Unit (GPU) takes weeks.

Finding a shorter representation without losing too much information is the goal of some preprocessing algorithms (Bell and Sejnowski 1997b, Krizhevsky and Hinton 2011, Farabet et al. 2013). However, with the help of a GPU, we can fully utilize the abstraction levels provided by CNNs and Deep Learning with complete images. In Farabet et al. (Farabet et al. 2013), horizontal flipping, rotation and scaling were used to decrease the bias that low-level features can bring to deep classification.

This is further explored in Simard et al. (Simard et al. 2003), where distortions and translations were used to expand the MNIST database and optimize classification in many algorithms. Finding appropriate features for image matching and classification is one of the challenges that each dataset poses to the task (Dollar et al. 2007), as distribution, characteristics and size influence the speed and quality of it. Studio and product images have few works in the literature, and preprocessing and featuremining on these is a relatively unexplored area.

The Deep Learning classifier we use is well known in image classification (Sigaud and Wilson 2007, Arel et al. 2010, Zeng et al. 2013). Optimal accuracy was reached in many competitions by Deep Learning related approaches, and the Large Scale Visual Recognition Challenge (ILSVRC) thrives with similar classification techniques. The specific architecture we use, the Convolutional Neural Network, has numerous applications and detailed papers and results (Ranzato et al. 2011, Wu et al. 2014, Le et al. 2011, Krizhevsky et al. 2012a, Le 2013).

## 3 METHODOLOGY

The methodology used on this work consists of some preprocessing techniques followed by the classification at the end of the pipeline. The preprocessing stage consists of four transformations that are detailed here: a. Affine transformations; b. Perspective transformations; c. Image whitening; d. Image normalization.

Affine and perspective transformations can be used to increase generality in image classifiers where a common approach is to alter the referential camera position to distort the image and

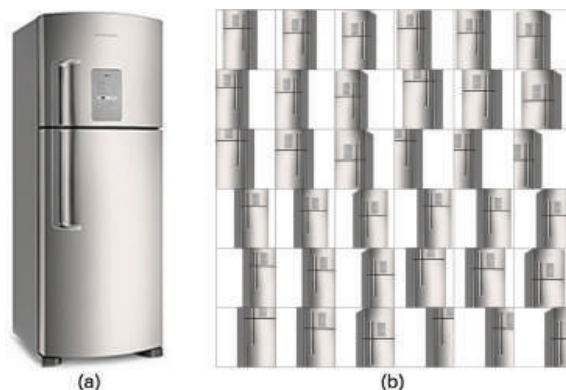


Figure 1. In (a): original image; In (b): 36 different images obtained with perspective transformations applied to the original image.

simulate different situations (Yaeger et al. 1996, Simard et al. 2003). Moreover, affine transformations keep distance ratios (midpoints of lines and their relative proportions are unaltered) and collinearity (points residing in lines keep their former positions on them). Composed by the translation, rotation, scaling and shearing operations, this group of transforms can focus on different parts of the image to increase the classification accuracy by a few percents. An example of perspective transformations can be viewed in Figure 1.

On the other hand, the perspective transformation projects points onto the image plane along lines that emanate from a single point, called the center of projection, changing the focus of the image. This type of transform does not preserve line parallelism, and is used to create distortions in images. These techniques are used in a variety of image classification tasks that involve low-level features (i.e. raw pixels) to increase the feature space and reduce bias.

In an image dataset, instances can have high contrast and illumination variation: this damages the classification process and can incur unnecessary bias. To soften the difference between images, matrix normalization and whitening algorithms can be applied before training. In Deep Learning, they are essential for optimization, but in non-DL algorithms these have little to no effect on classification.

Normalization is a process that decreases the variation in ranges between matrixes. In image processing, it is used to create consistency in datasets with natural scenes in a wide spectrum of illumination sources (Cirean et al. 2011, Makwana et al. 2011). Effectively, this process spreads out the pixel ranges on images to enhance contrast and edges, amplifying corners and form features. Many normalization techniques exist in computer vision. In this paper we use the Global Contrast Normalization (GCN), a common preprocessing approach to image classification (Goodfellow et al. 2013,

Springenberg and Riedmiller 2013). It consists in subtracting from each pixel intensity value the mean of all pixels in the image, then dividing by their standard deviation. Whitening algorithms are processes that decorrelate neighboring pixels by transforming the data until the covariance matrix is equal to the identity matrix. The pixels from an image are usually highly correlated, and thus many points are redundant in training and classification. Whitening is vital in the preprocessing step of algorithms that use raw images as data to enhance the identification of higher-order features.

In Bell et al. (Bell and Sejnowski 1997b, Bell and Sejnowski 1997a), several whitening techniques are discussed and explained. Mainly, they find a decorrelation matrix and multiply the desired data by it to transform the covariation matrix into the diagonal, then factoring it to produce the identity. Among them, the Principal Component Analysis (PCA) and the Zero-phase Component Analysis (ZCA) have been used in classification tasks by various authors.

The PCA is applied to a matrix  $X$  by using the following values from the covariance matrix: multiplying it by the transpose of the eigenvectors, scaled by the inverse square root of the eigenvalues. However, this transform is not unique as multiplying the whitened data by an orthogonal matrix will yield a result with covariance matrix equal to the identity. The ZCA consists in finding the canonical whitening by multiplying the PCA transform by an orthonormal matrix. This transform is unique and shows the closest resemblance to the original data. On the other hand, the ZCA consists in finding the canonical whitening by multiplying the PCA transform by an orthonormal matrix. This transform is unique and shows the closest resemblance to the original data. Both PCA and ZCA approaches can be viewed in Figure 2.

After all the preprocessing, the images are subjected to a Convolutional Neural Network (CNN) classifier. Neural networks have been used in several classification tasks (Hunt et al. 1992, Zhang

2000, Arel et al. 2010, Schmidhuber 2015), and the convolutional neural network in particular shows great results in visual classification. These hierarchical networks reduce the image by filtering and reducing it in each iteration, differing from the usual neural network approach by sharing weights between nodes in the same layer and spatial pooling for simplification.

A conventional Neural Network (NN) consists of many simple processors connected, called neurons, each one producing a sequence of values of activations. Input neurons are activated by sensors perceiving the environment, other neurons are activated by means of weighted connections from previous activated neurons. Some neurons can influence the network by firing actions. Learning or credit assignment is about finding weights that make the NN exhibit desired behavior, such as driving a car. Depending on the problem and how the neurons are connected, such behavior may require long causal chains of computational stages, where each stage transforms (often in a non-linear way) the aggregate activation of the network. Deep Learning is about accurately assigning credit across many such stages (Schmidhuber 2015).

A Convolutional Neural Network (CNN) has several different layers for each operation, but the basic model has a convolutional layer that separates the image in many sub-matrixes called feature maps, a pooling layer that groups pixels and an activation layer. These feature maps are each the result of a filter passed throughout the input matrix, and the activation layer can be chosen from a variety of functions, such as sigmoids or a Rectified Linear Unit. This network is fed with squared pixel patches surrounding each pixel in each image. The label for each patch is the label of the center pixel.

## 4 EXPERIMENTS

In our experiments, we used an image object database with roughly two million instances obtained from e-commerce sites and web crawlers. Numerous products, from kitchen appliances to tires and cell-phone accessories, were gathered to form it. To reduce the scope of classification, we restrained the size of this database to include only appliances and related items, totalling 10 classes. All images were standardized to  $256 \times 256$  pixels, with 3 color channels. To create a Convolutional Neural Network, we used the Deep Learning framework Caffe (Jia et al. 2014). The network layers are based on the reference CaffeNet (Krizhevsky et al. 2012b).

On each essay, we used a different preprocessing approach. First, we use the database without any augmentation. Then, a boosted database with affine transformations (rotations, translations

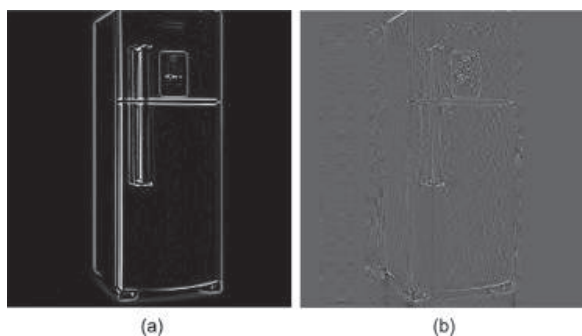


Figure 2. In (a): image after application of PCA transform; In (b): image after application of the ZCA transform.

with zoom and horizontal flipping) applied to all images. Finally, we use the ZCA whitening algorithm in all images in the boosted database and train with that. A fourth database containing other classes of objects was used to further measure the image augmented classification accuracy.

The classifiers are first tested for assertivity using a random sampling from the database. Then, with a second database composed of images mined with web crawlers and classified by hand, test the model for the top-1 and top-5 error rates. We separated resulting classifiers by their number of iterations, from 1000 to 5000 in 1000 steps. The first experiment was done in order to set a baseline for comparing our techniques results and has a test sampling set composed of 7,5% of the total images. On the following experiments this quantity was raised to 25%.

Tables 1, 3 and 5 refer to the training data and the accuracy results on each number of training iterations. The classifier takes snapshots of

Table 1. Classifier's performance for the training data.

| Original database—training data |             |
|---------------------------------|-------------|
| Iterations                      | Assertivity |
| 1000                            | 82.00%      |
| 2000                            | 87.02%      |
| 3000                            | 89.70%      |
| 4000                            | 90.75%      |
| 5000                            | 90.67%      |

Table 2. Classifier's performance for the testing data.

| Original database—testing data |        |        |
|--------------------------------|--------|--------|
| Iterations                     | Top-1  | Top-5  |
| 1000                           | 30.32% | 76.03% |
| 2000                           | 30.33% | 76.21% |
| 3000                           | 32.01% | 73.33% |
| 4000                           | 35.24% | 76.19% |
| 5000                           | 34.02% | 80.44% |

Table 3. Classifier's performance for the training data with boosted dataset.

| Boosted database—training data |             |
|--------------------------------|-------------|
| Iterations                     | Assertivity |
| 1000                           | 77.09%      |
| 2000                           | 83.00%      |
| 3000                           | 84.53%      |
| 4000                           | 86.00%      |
| 5000                           | 87.00%      |

itself and measures the error rate of the preliminar results in 1000, 2000, 3000, 4000 and 5000 iterations, giving the accuracy value (100%—error rate).

Tables 2, 4 and 6 refer to the test data taken from web crawlers and made of natural scenes. This test uses the snapshot classifiers that were obtained in the training phase and measures the error rate when choosing the prediction of the most likely classes. The top-1 error rate takes in account only one prediction and the top-5, a set with the five most probable classes.

Table 7 is an elaborate description of the results from the last classifier by class. Each class has a super-class (parent), the number that represents the class and the name of the class. We measured the error rates of the last classifier with 5000 iterations and separated the accuracy and recall per class.

Table 4. Classifier's performance for the testing data with boosted dataset. The gain in both top-1 and top-5 metrics is around 8%.

| Boosted database—testing data |        |        |
|-------------------------------|--------|--------|
| Iterations                    | Top-1  | Top-5  |
| 1000                          | 25.12% | 76.04% |
| 2000                          | 44.09% | 84.37% |
| 3000                          | 40.37% | 86.11% |
| 4000                          | 47.31% | 85.19% |
| 5000                          | 47.04% | 88.44% |

Table 5. Classifier's performance for the training data with boosted and whitened dataset.

| Boosted + ZCA database—training data |             |
|--------------------------------------|-------------|
| Iterations                           | Assertivity |
| 1000                                 | 75.09%      |
| 2000                                 | 83.26%      |
| 3000                                 | 85.48%      |
| 4000                                 | 86.33%      |
| 5000                                 | 87.29%      |

Table 6. Classifier's performance for the testing data with boosted and whitened dataset. A slight improvement was obtained.

| Boosted + ZCA database—testing data |        |        |
|-------------------------------------|--------|--------|
| Iterations                          | Top-1  | Top-5  |
| 1000                                | 39.25% | 89.19% |
| 2000                                | 42.92% | 88.31% |
| 3000                                | 47.70% | 94.17% |
| 4000                                | 52.18% | 90.95% |
| 5000                                | 44.43% | 92.42% |



Table 7. Detailed information describing accuracy and recall for Top-1 and Top-5 for each class.

| Boosted database with extra classes—detailed test data |       |       |        |       |
|--------------------------------------------------------|-------|-------|--------|-------|
| Class                                                  | Top-1 | Top-3 | Recall |       |
|                                                        |       |       | Top-1  | Top-3 |
| Tires                                                  | 0.78% | 0.78% | 1.00%  | 1.00% |
| Car mats                                               | 0.40% | 0.60% | 0.40%  | 0.60% |
| Wine cellars                                           | 0.73% | 0.82% | 0.82%  | 0.91% |
| Freezers                                               | 0.82% | 1.00% | 0.82%  | 0.91% |
| Microwaves                                             | 0.64% | 0.73% | 0.73%  | 1.00% |
| Dishwashers                                            | 0.55% | 0.73% | 1.00%  | 1.00% |
| Ovens                                                  | 0.45% | 0.82% | 0.73%  | 1.00% |
| Cooktops                                               | 0.50% | 0.50% | 0.38%  | 0.62% |
| Stoves                                                 | 0.30% | 0.40% | 0.90%  | 1.00% |
| Washmachines                                           | 0.00% | 0.18% | 0.64%  | 1.00% |
| Drywashers                                             | 0.45% | 0.73% | 1.00%  | 100%  |
| Dryers                                                 | 0.55% | 1.00% | 0.64%  | 1.00% |
| Centrifuges                                            | 0.30% | 0.60% | 0.90%  | 0.90% |
| Refrigerators                                          | 0.20% | 0.50% | 0.70%  | 1.00% |
| Watches                                                | 0.12% | 0.36% | 0.27%  | 0.55% |
| Suitcases                                              | 0.30% | 0.30% | 1.00%  | 1.00% |
| Keys                                                   | 0.64% | 0.64% | 1.00%  | 1.00% |

The first preprocessing stage led to significant increase in accuracy, but in the training of the last classifier for the database with the preprocessing plus the ZCA transformations the classifier did not converge in the maximum 5000 iterations. Testing the same approach with a database composed of natural images, however, yielded better results than the second classifier.

## 5 CONCLUSIONS

In our experiments, we first evaluated the classifiers performance against a image dataset without any preprocessing. In this scenario, we obtained an accuracy of 80% within the top-5 predictions, which means that one of the 5 best predictions yielded by the classifier for a image is correct. On the other hand, for the top-1, the classifier performed with 34% accuracy. By training the classifier again with the preprocessing techniques described earlier, we were able to improve the classification to a 92% accuracy on the top-5 and 44% accuracy on the top-1. This means an improvement of 15% in the top-5 and 30% in the top-1 predictions.

The first and second classifiers yielded results as expected, as primary preprocessing algorithms such as affine transformations do not heavily depend on the color composition of the image and can give the classifier a good amount of generalization. But the third classifier did not even converge

in the maximum number of iterations previously set, without any tangible results.

The dataset without studio images reached great accuracy rating. We state here that studio images cannot be doctored with the current famous preprocessing techniques and increase a Convolutional Neural Network classifier accuracy directly: extra steps must be made to ensure that the white background does not interfere.

Some possible ways of minimizing the background's importance in the picture are the use of image segmentation algorithms such as selective search (Sande 2011, Uijlings et al. 2013) or DPM (Felzenszwalb et al. 2010, Trulls et al. 2014) to extract only the object from the image and create features from it; Or inserting dummy backgrounds from natural images behind objects. This can lead to some bias if the background list is small; Also, edge detection filters may eliminate some of the white interference, as regions with that background will have no edges and no useful information. Illumination will also be discarded. Another improvement is the creation of a tree of classifiers where the top most classifiers are more generic as the bottom most classifiers are more specific. Also, using binary classifiers for each class in conjunction with the main classifier as a pool can add to the generality of the solution.

We first evaluated the classifiers performance against a image dataset without any preprocessing. In this scenario, we obtained an accuracy of 80% within the top-5 predictions, which means that one of the 5 best predictions yielded by the classifier for a image is correct. On the other hand, for the top-1, the classifier performed with 34% accuracy. By training the classifier again with the preprocessing techniques described earlier, we were able to improve the classification to a 92% accuracy on the top-5 and 44% accuracy on the top-1. This means an improvement of 15% in the top-5 and 30% in the top-1 predictions.

The first and second classifiers yielded results as expected, as primary preprocessing algorithms such as affine transformations do not heavily depend on the color composition of the image and can give the classifier a good amount of generalization. But the third classifier did not even converge in the maximum number of iterations previously set, without any tangible results. The dataset without studio images reached great accuracy rating. We state here that studio images cannot be doctored with the current famous preprocessing techniques and increase a Convolutional Neural Network classifier accuracy directly: extra steps must be made to ensure that the white background does not interfere.

Some possible ways of minimizing the background's importance in the picture are the use of image segmentation algorithms such as selective

search (Sande 2011, Uijlings et al. 2013) or DPM (Felzenszwalb et al. 2010, Trulls et al. 2014) to extract only the object from the image and create features from it; Or inserting dummy backgrounds from natural images behind objects. This can lead to some bias if the background list is small; Also, edge detection filters may eliminate some of the white interference, as regions with that background will have no edges and no useful information. Illumination will also be discarded. Another possible improvement that we are currently working on is the creation of a tree of classifiers where the top most classifiers are more generic and the following classifiers on the tree are more specific. Also, using binary classifiers for each class in conjunction with the main classifier as a pool can add to the generality of the solution.

## REFERENCES

- Arel, I., D.C. Rose, & T.P. Karnowski (2010). Deep Machine Learning -A New Frontier in Artificial Intelligence Research [Research Frontier].
- Bell, A. & T. Sejnowski (1997a). Edges are the “independent components” of natural scenes. *Advances in neural information processing systems*, 831–837.
- Bell, A.J. & T.J. Sejnowski (1997b). The ‘independent components’ of natural scenes are edge filters. *Vision Research* 37(23), 3327–3338.
- Cirean, D., U. Meier, & J. Schmidhuber (2011). A committee of neural networks for traffic sign classification. In *Proceedings of the International Joint Conference on Neural Networks*, pp. 1918–1921.
- Deng, J., W. Dong, R. Socher, L.-J. Li, K. Li, & L. Fei-Fei (2009). ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.
- Dollar, P., Z. Tu, H. Tao, & S. Belongie (2007). Feature mining for image classification. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*.
- Farabet, C., C. Couprie, L. Najman, & Y. Lecun (2013). Learning hierarchical features for scene labeling. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35(8), 1915–1929.
- Felzenszwalb, P.F., R.B. Girshick, & D. McAllester (2010). Cascade object detection with deformable part models. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 2241–2248.
- Förstner, W. (2000). Image Preprocessing for Feature Extraction in Digital Intensity, Color and Range Images. In *Geomatic Method for the Analysis of Data in the Earth Sciences*, Volume 95, pp. 165–189.
- Fuxman, A., A. Kannan, A.B. Goldberg, R. Agrawal, P. Tsaparas, & J. Shafer (2009). Improving classification accuracy using automatically extracted training data. *Proceedings of the 15th ACM International Conference on Knowledge Discovery and Data Mining* (2009) 33(2), 1145.
- Goodfellow, I.J., D. Warde-Farley, M. Mirza, A. Courville, & Y. Bengio (2013). Maxout Networks. *arXiv preprint*, 1319–1327.
- Graham, N. & A. Sutter (2000). Normalization: Contrast-gain control in simple (Fourier) and complex (non-Fourier) pathways of pattern vision.
- Hunt, K.J., D. Sbarbaro, R. Zbikowski, & P.J. Gawthrop (1992). Neural networks for control systems—A survey.
- Jia, Y., E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, & T. Darrell (2014). Caffe: Convolutional architecture for fast feature embedding. *arXivpreprintarXiv:1408.5093*.
- Kotsiantis, S.B. & D. Kanellopoulos (2006). Data preprocessing for supervised learning. *International Journal of ...*, 1–7.
- Krizhevsky, A. & G. Hinton (2011). Using Very Deep Autoencoders for Content-Based Image Retrieval. *ESANN*.
- Krizhevsky, A., I. Sutskever, & G.E. Hinton (2012a). ImageNet Classification with Deep Convolutional Neural Networks. *Advances In Neural Information Processing Systems*, 1–9.
- Krizhevsky, A., I. Sutskever, & G.E. Hinton (2012b). Imagenet classification with deep convolutional neural networks. In F. Pereira, C. Burges, L. Bottou, and K. Weinberger (Eds.), *Advances in Neural Information Processing Systems 25*, pp. 1097–1105. Curran Associates, Inc.
- Le, Q.V. (2013). Building high-level features using large scale unsupervised learning. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, pp. 8595–8598.
- Le, Q.V., R. Monga, M. Devin, G. Corrado, K. Chen, M.A. Ranzato, J. Dean, & A.Y. Ng (2011). Building high-level features using large scale unsupervised learning Dataset constructions. *ICML*, 1–10.
- LeCun, Y., L. Bottou, Y. Bengio, & P. Haffner (2001). Gradient-based learning applied to document recognition. In *Intelligent Signal Processing*, pp. 306–351. IEEE Press.
- Lecun, Y. & C. Cortes. The MNIST database of hand-written digits.
- Makwana, R., V. Thakar, & N. Chauhan (2011, Nov). Evaluation and analysis of illumination normalization methods for face recognition. In *Image Information Processing (ICIIP), 2011 International Conference on*, pp. 1–6.
- Ranzato, M., Q.V. Le, R. Monga, M. Devin, K. Chen, G.S. Corrado, J. Dean, & A.Y. Ng (2011). Building high-level features using large scale unsupervised learning. In *International Conference in Machine Learning*, pp. 38115.
- Russakovsky, O., J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A.C. Berg, & L. Fei-Fei (2014). Imagenet large scale visual recognition challenge.
- Russell, B., A. Torralba, K. Murphy, & W. Freeman (2008). Labelme: A database and web-based tool for image annotation. *International Journal of Computer Vision* 77(1–3), 157–173.
- Sande, K.V.D. (2011). Segmentation as selective search for object recognition. ... *Vision (ICCV), 2011...*(2), 1879–1886.
- Schmidhuber, J. (2014). Deep Learning in Neural Networks: An Overview. *arXivpreprintarXiv:1404.7828*, 1–66.

- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks* 61, 85–117. Published online 2014; based on TR arXiv:1404.7828 [cs.NE].
- Sermanet, P., S. Chintala, & Y. LeCun (2012). Convolutional neural networks applied to house numbers digit classification. *Proceedings of International Conference on Pattern Recognition ICPR12*, 10–13.
- Sigaud, O. & S.W. Wilson (2007). Learning classifier systems: A survey. *Soft Computing* 11(11), 1065–1078.
- Simard, P., D. Steinkraus, & J. Platt (2003). Best practices for convolutional neural networks applied to visual document analysis. *Seventh International Conference on Document Analysis and Recognition, 2003. Proceedings.*
- Smirnov, E.A., D.M. Timoshenko, & S.N. Andrianov (2014). Comparison of Regularization Methods for ImageNet Classification with Deep Convolutional Neural Networks. *AASRI Procedia* 6, 89–94.
- Springenberg, J. & M. Riedmiller (2013). Improving Deep Neural Networks with Probabilistic Maxout Units. *arXiv:1312.6116*, 1–10.
- Trulls, E., S. Tsogkas, I. Kokkinos, A. Sanfeliu, & F. Moreno-Noguer (2014). Segmentation-aware deformable part models. In *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, pp. 168–175. IEEE.
- Uijlings, J.R.R., K.E.A. Van De Sande, T. Gevers, & A.W.M. Smeulders (2013). Selective search for object recognition. *International Journal of Computer Vision* 104(2), 154–171.
- Wu, C., W. Fan, Y. He, J. Sun, & S. Naoi (2014). Handwritten Character Recognition by Alternately Trained Relaxation Convolutional Neural Network. *2014 14th International Conference on Frontiers in Handwriting Recognition*, 291–296.
- Yaeger, L., R. Lyon, & B. Webb (1996). Effective Training of a Neural Network Character Classifier for Word Recognition. *Nips* 9, 807–813.
- Zeng, X., W. Ouyang, & X. Wang (2013). Multi-stage Contextual Deep Learning for Pedestrian Detection. *2013 IEEE International Conference on Computer Vision*, 121–128.
- Zhang, G.P. (2000). Neural networks for classification: a survey. In *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, Volume 30, pp. 451–462.